

TRI-CA EXPERIMENTAL PLATFORM

Installation & Configuration Tutorial

A practical guide for researchers setting up human-AI interaction experiments with TRI-CA: Trials with Conversational Agents.

Repository: <https://github.com/ProfNaama/TRI-CA>

Primary use case: controlled experiments with configurable prompts, questionnaires, treatment groups, and structured result storage.

This tutorial reflects the current TRI-CA repository structure and runtime behavior. It replaces older guidance that referred to outdated folders, analysis scripts, or deployment assumptions no longer present in the repo.

1. System Overview

TRI-CA is a Node.js and Express platform for controlled studies of human-AI interaction. It supports participant code entry, informed consent, pre-task questionnaires, a configurable chat task, post-task questionnaires, and structured data storage in one workflow.

Researchers configure most studies through CSV files plus prompt and task-description files, which makes it possible to run many experiments without modifying the application code.

Component	Description
Participant Interface	Code entry, consent, questionnaires, and chat UI served from the web app.
Application Server	Express server handling routing, session state, treatment assignment, and persistence.
LLM Integration	Built-in OpenAI chat-completions integration with a fake-response mode for local testing.
Configuration Layer	CSV files plus prompt and task banks that define the study flow and condition-specific behavior.
Storage Layer	Completed-session output to PostgreSQL, a local results file, or both.

Key idea: treatment-group-specific flow is configured through files. Each group can point to its own hidden prompt, task description, pre-question blocks, and post-question blocks.

2. Repository Structure

The current TRI-CA repository is a single Node.js app. The main folders and files are:

```
TRI-CA/  
app.js  
config.js  
helpers.js  
sessionManagement.js  
package.json  
README.md  
  
experiment_configuration/  
  experiment_desc.csv  
  treatment_groups_config.csv  
  questions_bank.csv  
  hidden_prompts_bank/  
  user_tasks_bank/  
  
views/  
  welcome_code.pug  
  consent.pug  
  user_questionnaire.pug  
  chat.pug  
  session_ended.pug  
  layout.pug  
  
static/  
  stylesheets/style.css  
  images/avatars/  
  fonts/  
  
pre_requisites/  
  generate_codes.py  
  postgres_table.txt
```

Older folder names such as `server/`, `frontend/`, `analysis/`, `prompt_bank/`, or `task_bank/` do not match the current repo and should not be used when setting up the system.

3. Requirements and Dependencies

3.1 Core software

Dependency	Recommended version	Notes
Node.js	v18 or later	Required to run the application.
npm	v9 or later	Used to install server dependencies.
Git	Current version	Used to clone the repository and manage updates.
Python	v3.9 or later	Only needed if you want to use the helper script for generating participant access codes.
PostgreSQL*	Current supported version	Recommended for live studies and required for database-backed code validation and result storage.

* Not required for a local smoke test if you use `REUSABLE_CODE=tri-ca` and `RESULTS_FILE`.

3.2 External services

- OpenAI API access is required for live model responses in the default integration.
- For local testing, you can skip live API calls by setting `FAKE_LLM_RESPONSE` instead of `OPENAI_API_KEY`.

PostgreSQL note: many first-time users only want to verify the experiment flow on their own machine. For that purpose, PostgreSQL is optional. Use the local smoke test first, then connect PostgreSQL when you are ready for code validation and persistent study storage.

4. Installation and Local Smoke Test

Step 1: Clone the repository

```
git clone https://github.com/ProfNaama/TRI-CA.git
cd TRI-CA
```

Step 2: Install dependencies

```
npm install
```

Step 3: Run a local smoke test

The fastest way to verify TRI-CA is to run it locally without PostgreSQL and without calling OpenAI. The following command starts the server with a reusable local test code and a fixed fake assistant reply:

```
REUSABLE_CODE=tri-ca \
FAKE_LLM_RESPONSE="This is a fake TRI-CA response for local testing." \
RESULTS_FILE=/tmp/results.txt \
node app.js
```

Step 4: Test the participant flow

1. Open `http://localhost:3030`.
2. Enter the access code `tri-ca`.
3. Walk through consent, the questionnaire pages, the chat stage, and completion.
4. Inspect `/tmp/results.txt` to confirm that a completed session was saved.

Why this matters: this local smoke test catches most configuration errors before you connect external services, deploy to a public server, or invite participants. For a local run with participant-specific codes, see the participant access-code section below.

5. Environment Variables and LLM Setup

Required for a live study

```
OPENAI_API_KEY=your_api_key_here  
DATABASE_URL=postgresql://user:password@host:port/database
```

Common optional variables

Variable	Purpose
PORT	Server port. Default: 3030.
RESULTS_FILE	Append completed sessions to a local file as JSON lines.
LOCAL_CODES_FILE	Validate participant codes from a local JSON file without PostgreSQL.
REUSABLE_CODE	Developer testing code that bypasses normal code validation.
OPENAI_MODEL	OpenAI model name. Default: gpt - 4o.
OPENAI_TOKEN_LIMIT	Maximum response length for OpenAI calls.
FAKE_LLM_RESPONSE	Fixed assistant reply used when no API key is configured.
COMPLETE_CODE / GENERATE_UNIQUE_COMPLETION_CODE	Controls the completion code shown to participants.
REDIRECT_URL	Optional redirect target shown on the completion page.

TRI-CA currently ships with built-in OpenAI integration. If you want to use another provider, you will need to adapt the backend LLM call in the Node.js application.

Security note: do not commit secrets to the repository. Keep environment variables in your host configuration or a local untracked file.

6. Database Setup and Result Storage

TRI-CA supports two main storage patterns:

- **Local results file:** good for testing and small pilot runs. Set `RESULTS_FILE` and completed sessions will be appended as JSON lines. If you also set `LOCAL_CODES_FILE`, TRI-CA can validate participant codes from a local JSON file.
- **PostgreSQL:** recommended for live studies, persistent result storage, and database-backed participant-code validation.

PostgreSQL schema

Use the schema in `pre_requisites/postgres_table.txt` to create the `tri_ca_codes` and `tri_ca_results` tables.

Participant access codes

The helper script below generates codes for a specific experiment ID:

```
python pre_requisites/generate_codes.py --expid your_experiment_id
```

By default, the helper generates 100 codes of length 10. Use `--seed 42` for reproducible output; otherwise it uses the current Unix timestamp. The default output prints three labeled sections: a CSV-like list, a PostgreSQL INSERT block, and a local JSON block suitable for `LOCAL_CODES_FILE`.

For a local run with participant-specific codes and no PostgreSQL, you can write the JSON output to a file and launch TRI-CA with it:

```
python pre_requisites/generate_codes.py --expid local_demo --format local-json > /tmp/tri_ca_local_codes.json
EXPERIMENT_ID=local_demo LOCAL_CODES_FILE=/tmp/tri_ca_local_codes.json
FAKE_LLM_RESPONSE="This is a fake TRI-CA response for local testing."
RESULTS_FILE=/tmp/results.txt node app.js
```

For live studies, insert the generated codes into `tri_ca_codes` before launch. During a local smoke test, you can skip this step and use `REUSABLE_CODE=tri-ca`.

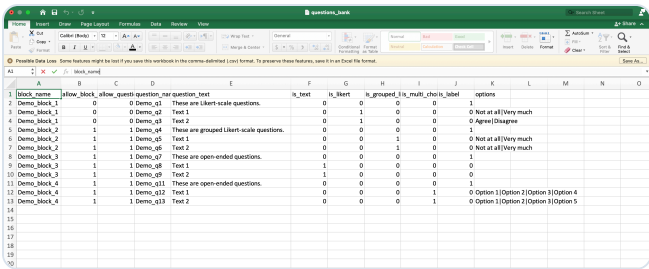
What gets saved

- Session identifiers and timestamps
- Treatment-group assignment
- Pre-questionnaire and post-questionnaire responses
- Conversation transcripts and per-turn interaction times
- Completion code and redirect information, if configured

7. Experiment Configuration Files

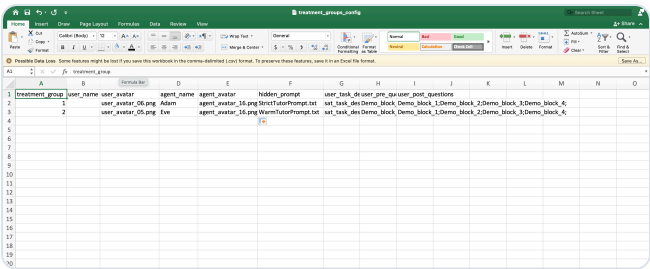
TRI-CA studies are primarily configured through three CSV files plus two file banks:

File or folder	Purpose
experiment_configuration/questions_bank.csv	Defines questionnaire items, question types, labels, and block-level randomization rules.
experiment_configuration/experiment_desc.csv	Controls selected page titles, headers, and body copy for experiment stages.
experiment_configuration/treatment_groups_config.csv	Defines treatment groups, prompt files, task files, avatars, and questionnaire blocks.
experiment_configuration/hidden_prompts_bank/	Stores plain-text hidden prompt files.
experiment_configuration/user_tasks_bank/	Stores HTML fragments shown as participant task descriptions.



A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	block_name	allow_block	allow_question	question_text	is_text	is_likert	is_grouped	is_multi	choice_label	options				
2	Demo_block_1	0	0	Demo_q01 These are Likert-scale questions.	0	0	0	0	0	0				
3	Demo_block_1	0	0	Demo_q01 Test 1	0	1	0	0	0	0	Not at all	Very much		
4	Demo_block_1	0	0	Demo_q01 Test 2	0	1	0	0	0	0	Agree	Disagree		
5	Demo_block_2	1	1	Demo_q01 These are grouped Likert-scale questions.	0	0	0	0	0	1				
6	Demo_block_2	1	1	Demo_q01 Test 1	0	0	1	0	0	0	Not at all	Very much		
7	Demo_block_2	1	1	Demo_q01 Test 2	0	0	1	0	0	0	Not at all	Very much		
8	Demo_block_3	1	1	Demo_q01 These are open-ended questions.	0	0	0	0	0	1				
9	Demo_block_3	1	1	Demo_q01 Test 1	1	0	0	0	0	0				
10	Demo_block_3	1	1	Demo_q01 Test 2	1	0	0	0	0	0				
11	Demo_block_4	1	1	Demo_q01 These are open-ended questions.	0	0	0	0	0	1				
12	Demo_block_4	1	1	Demo_q01 Test 1	0	0	0	1	0	0	Option 1	Option 2	Option 3	Option 4
13	Demo_block_4	1	1	Demo_q01 Test 2	0	0	0	1	0	0	Option 1	Option 2	Option 3	Option 4

Question bank example



A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	treatment_group	user_name	user_avatar	agent_name	agent_avatar	hidden_prompt	user_task	des	user_pre_questions					
2	1	user_avatar_06.png	Adem	agent_avatar_16.png	DirectTaskPrompt.txt	user_task_des	Demo_block_1	Demo_block_2	Demo_block_3	Demo_block_4				
3	2	user_avatar_05.png	Eve	agent_avatar_16.png	DirectTaskPrompt.txt	user_task_des	Demo_block_1	Demo_block_2	Demo_block_3	Demo_block_4				

Treatment-group configuration example

In the default UI, supported question types are text input, single-item Likert, grouped Likert, single-select multiple choice, and label rows.

8. Deployment, Live Launch, and Participant Flow

Local vs live: for a local smoke test or a small ad hoc study on one machine, use the local options described earlier. This section assumes a live or public deployment.

Deployment options

TRI-CA can run on any Node.js-compatible host that exposes environment variables and a public URL. Common options include:

- Heroku or another platform-as-a-service
- AWS, Azure, or GCP
- An institutional research server
- A lab-managed machine for internal pilots

Live launch checklist

- Configure the experiment files under `experiment_configuration/`.
- Create the PostgreSQL tables and load participant codes.
- Set `OPENAI_API_KEY`, `DATABASE_URL`, `EXPERIMENT_ID`, and `SESSION_SECRET`.

```
OPENAI_API_KEY=your_openai_api_key_here \  
DATABASE_URL=postgresql://user:password@host:port/database \  
EXPERIMENT_ID=your_experiment_id \  
SESSION_SECRET=replace_with_a_long_random_secret \  
PORT=8080 \  
node app.js
```

Deployment limitation: the default application uses the in-memory Express session store. For live studies, prefer a single running server unless you redesign session storage.

Participant flow

1. Participant enters an access code.
2. The platform validates the code against PostgreSQL or `LOCAL_CODES_FILE`, unless a reusable local test code is configured.
3. A session is initialized and assigned to a treatment group.
4. The participant completes consent.
5. Configured pre-questionnaire blocks are shown.
6. The participant completes the chat task with the treatment-specific hidden prompt and task instructions.
7. Configured post-questionnaire blocks are shown.
8. The completed session is saved and the participant may receive a completion code or redirect button.

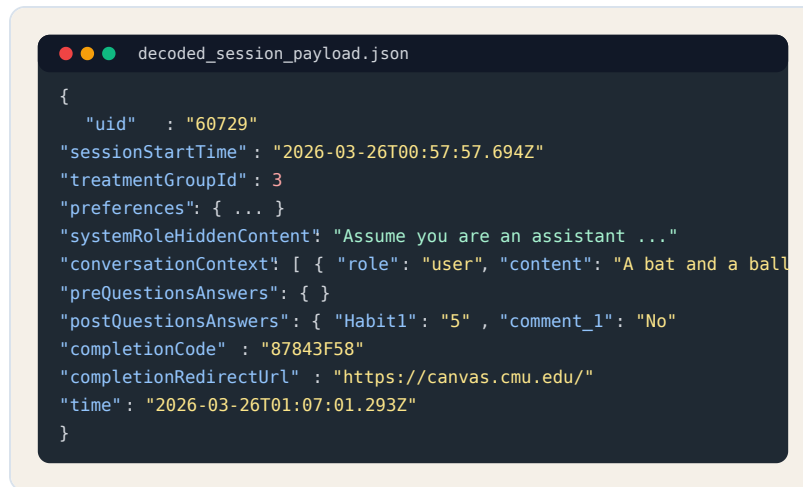
If a treatment group has no configured pre- or post-question blocks, the corresponding stage is skipped automatically. If a treatment group has a blank hidden prompt, the chat stage is skipped automatically.

9. Output Data, Testing, and Limitations

Output data

Depending on your setup, completed sessions can be written to RESULTS_FILE, PostgreSQL, or both. The saved data include treatment-group assignment, questionnaire responses, conversation history, timestamps, and optional completion metadata.

- `uid` and `treatmentGroupId` identify the session and assigned condition.
- `preQuestionsAnswers` and `postQuestionsAnswers` store questionnaire responses.
- `conversationContext` stores the participant-assistant exchange.
- `time`, `completion-code` fields, and `redirect` fields capture end-of-session metadata when configured.



Example decoded session payload structure

Testing checklist

- Run the local smoke test before using a public host.
- Verify questionnaire pages and treatment-group file paths.
- Verify PostgreSQL writes and participant-code validation before launch.
- Complete at least one full participant run end to end.

Known limitations

- Incomplete sessions are not saved.
- Participant codes are marked completed only at the final submission step, so a participant who leaves early and later starts a new session can reuse the same code.
- Malformed CSV files can break startup or runtime behavior.
- `question_name` values should be globally unique.
- Answer-based conditional branching is not implemented in the default questionnaire renderer.
- Existing code validation is simple and research-oriented, not intended as a hardened authentication system.

Recommendation: use the local smoke test first, then connect PostgreSQL, then deploy to a public host only after you have validated the full participant flow.